

Bank Fraud Watch: Deterministic Fraud Detection with Human-Confirmed Community Intelligence for Small-Business Banking

Bizer Engineering · Miles Agent Platform · gobizer.com

ABSTRACT

Small-business owners rarely reconcile their bank feeds daily, so card fraud is typically discovered weeks late — or never. We present **Bank Fraud Watch**, a fraud detection system built into the Bizer business platform that monitors bank transactions synced through Plaid, raises explainable alerts through notifications, email, and a conversational AI agent (Miles), and routes every verdict through the account owner. Detection is fully deterministic: five behavioral rules with account-relative thresholds run nightly over the normalized ledger, with no model or LLM in the data path. Owner-confirmed fraud is published as an anonymized vendor fingerprint to a shared community registry, which both the agent and external consumers (via a read-only API) can query — so each confirmed report hardens every other business on the platform. Calibrating the rules against 2.65 million labeled transactions from a public fraud-detection training corpus reduced alert volume by 10× while raising transaction-level precision from 2.6% to 24.8%, demonstrating that public machine-learning datasets can serve as effective calibration corpora even for rule-based systems. We describe the architecture, the detection methodology, the privacy model of the community registry, and the evaluation harness, and discuss limitations and the path toward a learned scoring layer trained on the platform's own resolved alerts.

1 Introduction

Payment fraud disproportionately harms small businesses: they carry consumer-grade exposure without a bank's fraud desk, a finance team, or time to reconcile statements. The median owner checks their transactions on their

phone, occasionally, after the fact. By the time a fraudulent charge is noticed, the dispute window may be closing and the same card may have been drained repeatedly.

Bizer is a mobile-first business platform in which an AI agent, **Miles**, works alongside the owner. When an owner links their bank account through Plaid [6], Bizer already syncs transactions nightly into a normalized money ledger. Bank Fraud Watch extends that pipeline with three capabilities:

- **Detection** — a deterministic nightly scan over the bank feed that flags the behavioral signatures of card fraud;
- **Alerting and triage** — notifications, alert email, and a guided conversation in which Miles walks the owner through each alert, with the owner delivering the verdict;
- **Community intelligence** — an anonymized registry of owner-confirmed fraud, queryable by every business on the platform and, through a public API, by outside consumers.

Three design commitments shape the system. First, *no model in the data path*: detection is rule-based and reproducible, matching the platform's standing principle that its data pipelines are deterministic and the LLM only reads results at conversation time. Second, *the human owns the verdict*: the system flags, but only the owner classifies a charge as fraud, and that action is gated behind an explicit approval tap. Third, *alert scarcity is a feature*: an alert feed that cries wolf trains owners to ignore the alert that matters, so the rules are tuned for precision over recall — a choice we quantify in Section 6.

2 System Architecture

Bank Fraud Watch is strictly opt-in twice over: a business must both enable the `bank-fraud-watch` skill and have Plaid connected. Figure 1 shows the pipeline.

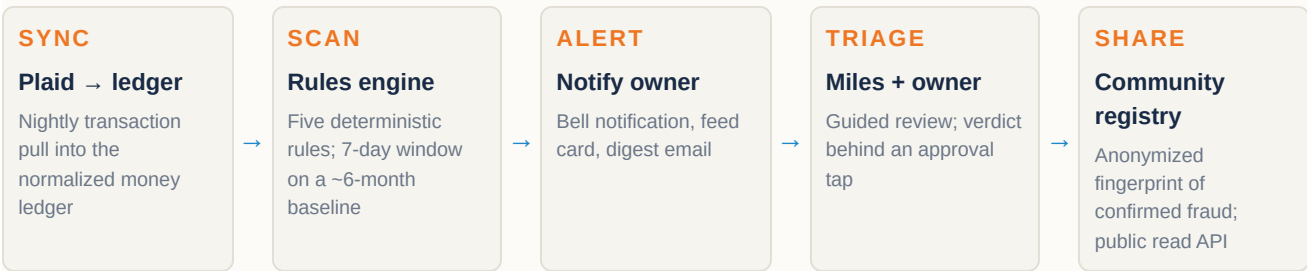


Figure 1. The Bank Fraud Watch pipeline. Everything left of triage is deterministic and runs unattended; everything that writes outside the business — the community fingerprint — requires the owner's explicit confirmation.

The scan runs after the nightly money sync so it always sees the freshest rows. Each business's scan loads roughly 190 days of bank-side ledger history: the most recent seven days form the *scan window*, and the preceding months form the *baseline* against which normality is defined. Alerts are persisted per business with a lifecycle of `open` →

confirmed | legitimate ; a vendor the owner marks legitimate is muted in all future scans, closing the feedback loop.

Every alert carries a **stable identifier** derived from its rule and anchoring transaction. Re-scans therefore re-derive the same identifiers and create nothing new — idempotency falls out of the ID scheme rather than bookkeeping, and a re-created alert can never re-notify the owner.

3 Detection Methodology

Ledger rows are minimal by design: a timestamp, a positive amount with a separate direction flag, a vendor string, and a category. There is no card-network metadata — no merchant category codes, no geolocation, no authorization trail. The rules are therefore purely behavioral, keyed on the account's own history. Vendor strings are normalized into match keys (case, punctuation, and store numbers removed, so **STARBUCKS #1234** and **Starbucks 0088** collide).

RULE	TRIGGER (OUTFLOWS ONLY)	SEVERITY
duplicate-charge	Same vendor and same amount, twice or more within 72 hours	medium; high ≥ \$100
card-testing	≥ 4 charges of ≤ \$15 inside one hour, all at vendors the account has <i>never</i> used	high
new-vendor-large	Vendor absent from the baseline, amount ≥ max(\$200, 3× median debit, 2× the account's p95 debit)	medium; high ≥ \$1,000
amount-spike	Vendor with ≥ 5 prior charges and a stable pattern (max ≤ 3× median) charging ≥ 6× its historical max, ≥ \$100 and ≥ 2× the account's p95	medium; high ≥ \$500
velocity-spike	24-hour outflow ≥ 2× the worst baseline day and ≥ \$500	medium

Table 1. The five detection rules. Every threshold shown was either validated or introduced by the calibration study in Section 6.

Two ideas do most of the work. The first is **account-relative thresholds**: both amount rules require a charge to be an outlier for *this account* — at least twice its own 95th-percentile debit — rather than merely exceeding a fixed dollar floor. A \$400 charge is an anomaly for a food truck and routine for a contractor; hard floors cannot express that, and in calibration they drowned real fraud in ordinary variance. The second is **pattern maturity**: a "spike" is

only meaningful against a vendor with real, stable history (five or more charges whose maximum stays within 3× their median), and "novelty" is only suspicious when the account has enough baseline for novelty to mean anything. Each gate encodes a fact about how evidence works, which is why they transfer across business sizes even though they were tuned on consumer data.

Every signal carries an owner-readable *reason* — the sentence Miles and the alert email lead with. Explainability is not a presentation layer; it is the contract that lets a non-expert owner deliver the verdict the whole system depends on.

4 Alerting and Human-in-the-Loop Triage

New alerts fan out through three channels: a deduplicated in-app notification per alert, a self-replacing feed card summarizing everything open, and one digest email per scan run. All three routes lead to the same place — a conversation with Miles.

The agent's triage protocol is deliberately conservative. Miles lists open alerts with their reasons, jogs the owner's memory first (a duplicate is often a retry; a new vendor is often just a new supplier), consults the community registry for the vendor in question, and then records the owner's verdict. That final write is gated: the tool call queues an approval card, and only the owner's tap executes it. The agent's instructions also encode honesty rules — it never labels a charge fraud on its own authority, never claims the platform blocked or reversed anything (it cannot), treats an empty registry lookup as "no reports yet" rather than "safe," and always routes confirmed fraud toward the owner's bank dispute process, offering to draft the dispute note.

A verdict of **legitimate** mutes the vendor for future scans. A verdict of **confirmed** is irreversible by design — a resolved alert cannot be flipped — because confirmation has an external consequence: publication to the community registry.

5 The Community Registry and External API

Individually, each business sees too little fraud to learn from. Collectively, the platform sees the same bad actors again and again. The community registry aggregates owner-confirmed fraud into per-vendor **fingerprints** that are anonymized by construction (Table 2): the registry stores nothing that could identify the reporting business, the account, or the transaction.

FIELD	CONTENTS	DELIBERATELY ABSENT
Vendor	Normalized vendor key and display name	—
Amount band	Coarse bucket: <\$10, \$10–50, \$50–200, \$200–1k, \$1k+	Exact amount
Pattern	Rule identifier and severity that caught it	Transaction record, description, account
Reporter	Salted hash, used only to count distinct reporters	Business identity; hashes are stripped from all reads
Counters	Report count, distinct reporters, first/last reported	—

Table 2. The anonymized fingerprint schema. Recent reports are capped per vendor; aggregate counters carry the long-term history.

Inside the platform, Miles queries the registry during triage — "three other businesses confirmed fraud from this vendor" changes an owner's prior substantially. Outside it, a read-only HTTPS API exposes the same fingerprints to banks, platforms, and researchers:

```
GET /fraudRegistryApi?vendor=<name>           # one vendor's fingerprint
GET /fraudRegistryApi?recent=1&limit=25       # latest reported vendors
x-api-key: <key>                             # hashed, server-side key registry
```

Because only human-confirmed reports enter the registry, its precision is structurally high — the registry inherits the owner's judgment, not the detector's. This is the deliberate inverse of feeds built from automated flags, and it is what makes the data worth consuming externally.

6 Evaluation: Public Training Data as a Calibration Corpus

Labeled fraud data is the scarcest resource in this domain, and a young platform has essentially none of its own. Our observation is that public datasets built for *training machine-learning models* serve equally well as *calibration corpora for rules* — provided their features map onto the production data model.

Among the standard public corpora, most are unusable for this purpose: the ubiquitous ULB dataset [3] exposes only PCA-anonymized components, and IEEE-CIS [4] carries competition-use restrictions. The **Sparkov** family [1] is the exception — simulated card transactions whose columns (merchant, category, amount, timestamp, label) map one-to-one onto our ledger rows, published CC0. We built a benchmark harness that streams a Sparkov CSV, treats

each card as one business's bank feed, and replays it through the *production* rules engine exactly as the nightly scan would run it: weekly steps, the same window and lookback, stable-ID deduplication, and optionally the owner-muting feedback loop. It reports transaction-level precision and recall, per-rule precision, and the alert-level statistics an owner actually experiences. At a fraud prevalence of 0.36%, accuracy is meaningless; precision and recall are the only honest lenses.

CONFIGURATION	ALERTS / ACCOUNT	PRECISION	RECALL	ALERTS W/ FRAUD
Original thresholds (dev slice, 50 accounts)	168 (~2/week)	2.6%	48.8%	2.6%
Tuned rules (same slice)	16.5	13.8%	26.2%	13.2%
Tuned rules, full corpus (997 accounts, 2.65M txns)	15.2 (~0.85/mo)	24.8%	41.7%	25.5%

Table 3. Calibration results on the Sparkov corpus (18 months of activity per account). Rules were tuned on the 50-account slice and validated on the full, mostly unseen corpus, where precision improved — evidence the gates generalize rather than overfit.

The headline result is the trade: a **10× reduction in alert volume** and a **10× gain in precision** for roughly half the (synthetic) recall. For an owner-facing feed this is unambiguously the right exchange — at two alerts per week and 97% noise, the original configuration would have extinguished owner trust within a month. Every gate in Table 1 that mentions history depth, stability, or the account-relative p95 floor was introduced by this study; the calibration also revealed that our single most precise rule (`velocity-spike`) was firing too rarely, and loosening it was a free win.

Three caveats keep the numbers honest. Sparkov is *consumer* card data at roughly ten times a small business's transaction volume. Its generative fraud model contains no duplicate-charge or card-testing behavior, so those rules score near zero there by construction; we retain them because the patterns are well-documented in business banking, and duplicate detection is valuable even when the cause is merchant error rather than fraud. And absolute dollar thresholds do not transfer from consumer data — which is precisely why calibration produced account-relative gates instead of new dollar floors. The harness is a regression instrument and a relative-comparison tool, never a production performance claim.

7 Security and Privacy Model

The system's write authority is asymmetric on purpose. Alert documents are readable by the business but writable only by the server — a client-forged "confirmed" would poison the community registry's inputs, so the confirmation path exists only behind authenticated, owner-gated server code. The registry and its API keys live in fully client-

denied collections; all reads pass through code that strips reporter hashes. The background scan follows the platform's autonomy rule for headless agents: it touches only opted-in businesses, and its single externally visible write — the community fingerprint — is downstream of an explicit human approval.

Privacy in the registry is achieved by construction rather than policy: coarse amount bands, rule identifiers, and salted reporter hashes are the *only* per-report data collected, so there is nothing sensitive to leak, subpoena, or de-anonymize downstream.

8 Limitations

- **Nightly latency.** Detection trails the transaction by up to a day; a fast-moving drain can complete before the scan runs.
- **Thin feature space.** Without pending status, location, or merchant category codes, geographic-impossibility and pending-versus-posted rules are out of reach today.
- **Domain shift in evaluation.** All quantitative results come from synthetic consumer data; true production precision is unknown until resolved-alert volume accumulates.
- **Registry cold start.** The community registry's value compounds with confirmations and is minimal at launch.
- **Recall ceiling.** Precision-first tuning knowingly forgoes fraud that mimics normal behavior; the system is a tripwire, not a guarantee.

9 Future Work

The near-term roadmap keeps the deterministic contract while widening the funnel. Retaining Plaid's pending/location side channel enables new rule families. Plaid transaction webhooks would move scanning from nightly to near-real-time — the scan is already factored per business, so a webhook handler can invoke it directly. A metered LLM second pass could re-rank alert severity using what Miles knows about the business ("they cater events; a \$900 first-time party-supply charge is probably real") without ever creating alerts on its own.

The benchmark harness is also the first half of a training pipeline: the same feature extraction can feed a small supervised model trained offline on labeled corpora and distilled to plain, deterministic code that *scores* rule-flagged alerts. The long-term training set that actually matches our domain, however, is the one the system began collecting on day one: every owner verdict is a labeled example on real small-business bank feeds. Public data is the bootstrap; the resolved-alert corpus — and the community registry built from it — is the moat.

10 References

1. Shenoy, K. *Credit Card Transactions Fraud Detection Dataset* (Sparkov). Kaggle, CC0. Generated with B. Harris's Sparkov Data Generation tool. kaggle.com/datasets/kartik2112/fraud-detection
2. Lopez-Rojas, E., Elmir, A., Axelsson, S. *PaySim: A financial mobile money simulator for fraud detection*. EMSS 2016.
3. Machine Learning Group, Université Libre de Bruxelles. *Credit Card Fraud Detection* dataset. kaggle.com/datasets/mlg-ulb/creditcardfraud
4. IEEE-CIS Fraud Detection. Kaggle competition dataset, Vesta Corporation, 2019.
5. Grover, P., et al. *Fraud Dataset Benchmark*. Amazon Science. github.com/amazon-science/fraud-dataset-benchmark
6. Plaid Inc. *Transactions API documentation*. plaid.com/docs/transactions
7. A Systematic Review of Machine Learning in Credit Card Fraud Detection Under Original Class Imbalance. *Computers* 14(10):437, MDPI, 2025. mdpi.com/2073-431X/14/10/437